

Project Title: Medicine Inventory Database Design & Implementation

MEDICINE INVENTORY MANAGEMENT SYSTEM

Database Systems Project Report

Submitted to:

Dr. Mohamed Mahmoud Elsayeh
Dr. Habiba Mohamed Ibrahim

Students:

Walaa Ezz Hassan – 241000009
Lujain Nabil Elsabagh – 242000024
Leen Hamdy Altamamy – 241000425
Mariam Sabry – 242000009

Course: CBIO204

Fundamentals of Databases

SYSTEM OVERVIEW

This project presents the design and implementation of a **Medicine Inventory Management System** using a relational database structure. The system is developed to efficiently manage and organize healthcare-related data in a structured and reliable way.

The database includes multiple interconnected entities such as medicines, diseases, doctors, prescriptions, prescription details, and billing records. These entities are linked through well-defined relationships to ensure data consistency, reduce redundancy, and maintain integrity across the system.

The system supports a **many-to-many relationship** between medicines and diseases, implemented using a junction table. It also includes relationships between doctors and prescriptions, as well as prescriptions and billing.

In addition, the system implements **SQL triggers** to automate important operations such as updating billing totals whenever prescription details are modified.

A set of **testing and analytical queries** is also included to verify the correctness of data insertion, relationships, and system functionality.

Overall, this project demonstrates a complete relational database solution that simulates a real-world healthcare inventory system with efficient data management and automation features.

DATABASE OVERVIEW

Database Name

medicine_inventory

Conceptual Design Overview

The database is designed based on a relational model that represents a Medicine Inventory Management System. It organizes healthcare data into multiple entities with clear relationships to ensure consistency and efficient data management.

The system includes the following main entities:

- Medicine
 - Disease
 - Doctor
 - Prescription
 - Prescription_Details (junction table)
 - Bill
 - Medicine_Disease (junction table)
-

Entities Summary

Medicine

Attributes: medicine_id, medicine_name, manufacture_date, expiry_date, price, disease_used_for, dosage

Disease

Attributes: disease_id, disease_name, disease_type

Doctor

Attributes: doctor_id, doctor_name, qualification, specialization

Prescription

Attributes: prescription_id, doctor_id, patient_name, issue_date

Prescription_Details

Attributes: detail_id, prescription_id, medicine_id, quantity

Description: This entity acts as a **junction table** that represents the many-to-many relationship between Prescription and Medicine.

Bill

Attributes: bill_id, prescription_id, tax, discount, total

Medicine_Disease

Attributes: medicine_id, disease_id

—

Design Concept

The system follows a relational database structure that supports:

- One-to-many relationships
- Many-to-many relationships using junction tables
- Data normalization to reduce redundancy
- Referential integrity using primary and foreign keys

—

Purpose of the Database

The main goal of this system is to manage medicines, diseases, prescriptions, and billing in a structured and efficient way while ensuring accurate relationships between all entities.

1. Medicine Entity Report

CONCEPTUAL DESIGN

Entity Name: Medicine

Description:

The Medicine entity stores information about medicines in the inventory system, including basic details such as name, manufacturing date, expiry date, price, dosage, and a disease reference field used for descriptive purposes.

ATTRIBUTES

- medicine_id
 - medicine_name
 - manufacture_date
 - expiry_date
 - price
 - disease_used_for
 - dosage
-

PRIMARY KEY

- medicine_id
-

CONSTRAINTS

- **PRIMARY KEY:** medicine_id
- **NOT NULL:** medicine_id, medicine_name, manufacture_date, expiry_date, price, dosage
- disease_used_for: optional attribute (**nullable**, not normalized)

RELATIONAL SCHEMA

```
MEDICINE(  
  medicine_id PK,  
  medicine_name,  
  manufacture_date,  
  expiry_date,  
  price,  
  disease_used_for,  
  dosage  
)
```

RELATIONSHIP DESCRIPTION

The Medicine entity is related to Disease and Prescription entities through **junction tables** to support **many-to-many relationships** and maintain **database normalization**.

DETAILED LINE-BY-LINE EXPLANATION (MEDICINE TABLE)

Line 1: **CREATE DATABASE IF NOT EXISTS medicine_inventory:**
Creates the database if it does not already exist to prevent duplication errors.

Line 2: **USE medicine_inventory:**
Selects the active database for all operations.

Line 3: **CREATE TABLE IF NOT EXISTS MEDICINE:**
Creates the **MEDICINE table** if it does not already exist.

Line 4: **medicine_id INT NOT NULL:**
Defines a unique identifier for each medicine and enforces **NOT NULL constraint**.

Line 5: **medicine_name VARCHAR(100) NOT NULL:**
Stores the medicine name as a required field.

Line 6: **manufacture_date DATE NOT NULL:**
Stores the manufacturing date of the medicine.

Line 7: **expiry_date DATE NOT NULL:**
Stores the expiry date used for validation and safety.

Line 8: **price DECIMAL(10,2) NOT NULL:**

Stores price with decimal precision for financial accuracy.

Line 9: **disease_used_for VARCHAR(100):**

Optional descriptive field; may cause **redundancy** since it is not normalized.

Line 10: **dosage VARCHAR(100) NOT NULL:**

Stores dosage instructions required for correct usage.

Line 11: **PRIMARY KEY (medicine_id):**

Ensures each record is uniquely identified using a **primary key constraint**.

Line 12: **INSERT IGNORE INTO MEDICINE VALUES:**

Inserts multiple records while ignoring duplicate entries.

Line 13–32: **Data rows:**

Each row represents a complete medicine record stored in the system.

SUMMARY

The Medicine entity is a core database component that ensures structured storage of medical data while maintaining **data integrity**, **normalization**, and proper relational design.

2.Disease Entity Report

CONCEPTUAL DESIGN

Entity Name: Disease

Description:

The Disease entity is designed to store information about diseases in the medicine inventory management system. Each disease contains a unique identifier, disease name, and disease classification type. The entity helps organize diseases into categories and improves database consistency and management.

ATTRIBUTES

- disease_id
 - disease_name
 - disease_type
-

PRIMARY KEY

- disease_id
-

DISEASE TYPES

- Infectious Disease
 - Deficiency Disease
 - Genetic Hereditary Disease
 - Non-Genetic Hereditary Disease
-

RELATIONAL SCHEMA

```
DISEASE(  
  disease_id PK,  
  disease_name,  
  disease_type  
)
```

CONSTRAINTS

- **PRIMARY KEY:** disease_id
 - **NOT NULL:** disease_id, disease_name, disease_type
 - **CHECK constraint:** disease_type must belong to predefined valid categories
-

RELATIONSHIP DESCRIPTION

The Disease entity has a **many-to-many (M:N) relationship** with the Medicine entity.
One medicine can treat multiple diseases, and one disease can be treated by multiple medicines.

DETAILED LINE-BY-LINE EXPLANATION (DISEASE TABLE)

Line 1: **DROP DATABASE IF EXISTS medicine_inventory;**
This command deletes the database if it already exists to avoid conflicts during redevelopment or testing.

Line 2: **CREATE DATABASE medicine_inventory;**
This creates a new database called medicine_inventory for the project.

Line 3: **USE medicine_inventory;**
This selects the database as the active working environment.

Line 4: **CREATE TABLE IF NOT EXISTS DISEASE (**
This creates the DISEASE table if it does not already exist.

Line 5: **disease_id INT NOT NULL,**
Defines disease_id as an integer and ensures it cannot be null.

Line 6: **disease_name VARCHAR(100) NOT NULL,**
Stores the name of the disease and ensures it is always provided.

Line 7: **disease_type VARCHAR(50) NOT NULL,**
Stores the classification type of the disease and ensures valid categorization.

Line 8: **PRIMARY KEY (disease_id)**
Defines disease_id as the primary key to uniquely identify each record.

Line 9: **);**
Closes the CREATE TABLE statement.

Line 10: **ALTER TABLE DISEASE**
Modifies the existing DISEASE table to add constraints.

Line 11: **ADD CONSTRAINT chk_disease_type**
Adds a named constraint to validate disease types.

Line 12: **CHECK (**
Starts the condition for validation.

Line 13: **disease_type IN (**
Defines allowed values for disease_type.

Line 14–17: **Allowed Disease Types**
These include Infectious, Deficiency, Genetic Hereditary, and Non-Genetic Hereditary diseases.

Line 18: **);**
Closes the CHECK constraint.

Line 19: **INSERT INTO DISEASE VALUES**
Begins inserting records into the table.

Line 20: **VALUES**
Specifies the data to be inserted.

Line 21 **onwards: Disease Records**
Each row represents a disease with its ID, name, and classification.

Final Line: **SELECT * FROM DISEASE;**
Retrieves all records from the DISEASE table for verification.

SUMMARY

The Disease entity is used to store and classify diseases in a structured way while maintaining **data integrity, validation rules**, and proper database organization.

3.Medicine_Disease Entity Report

CONCEPTUAL DESIGN

Entity Name: Medicine_Disease

Description:

The Medicine_Disease entity is a junction table used to represent the many-to-many (M:N) relationship between Medicine and Disease entities. It stores which medicines are used to treat which diseases in the system.

ATTRIBUTES

- medicine_id
 - disease_id
-

PRIMARY KEY

- (medicine_id, disease_id) (*Composite Primary Key*)
-

CONSTRAINTS

- **PRIMARY KEY:** (medicine_id, disease_id)
 - **FOREIGN KEY:**
 - medicine_id REFERENCES Medicine(medicine_id)
 - disease_id REFERENCES Disease(disease_id)
 - **NOT NULL:** medicine_id, disease_id
-

RELATIONAL SCHEMA

```
MEDICINE_DISEASE(  
  medicine_id PK/FK,  
  disease_id PK/FK  
)
```

RELATIONSHIP DESCRIPTION

This table represents a **many-to-many (M:N) relationship** between Medicine and Disease.

- One Medicine can be linked to many Diseases
- One Disease can be treated by many Medicines

The junction table eliminates redundancy and ensures proper normalization.

DETAILED LINE-BY-LINE EXPLANATION

Line 1: **CREATE TABLE IF NOT EXISTS MEDICINE_DISEASE;**
This creates the junction table if it does not already exist.

Line 2: **medicine_id INT NOT NULL;**
Stores the ID of the medicine and links to the Medicine table.

Line 3: **disease_id INT NOT NULL;**
Stores the ID of the disease and links to the Disease table.

Line 4: **PRIMARY KEY (medicine_id, disease_id);**
Defines a composite primary key to ensure each pair is unique.

Line 5: **FOREIGN KEY (medicine_id) REFERENCES Medicine(medicine_id);**
Ensures medicine_id exists in the Medicine table.

Line 6: **FOREIGN KEY (disease_id) REFERENCES Disease(disease_id);**
Ensures disease_id exists in the Disease table.

SUMMARY

The Medicine_Disease entity is a linking table that properly implements a **many-to-many relationship**, ensuring data consistency and normalization in the database.

4.Doctor Entity Report

CONCEPTUAL DESIGN

Entity Name: Doctor

Description: The Doctor entity stores information about doctors who diagnose patients and issue prescriptions for the appropriate medicines. It includes their ID, name, qualification, and specialization.

ATTRIBUTES

- doctor_id
 - doctor_name
 - qualification
 - specialization
-

PRIMARY KEY

- doctor_id
-

CONSTRAINTS

- **PRIMARY KEY:** doctor_id
 - **NOT NULL:** doctor_id
-

RELATIONAL SCHEMA

```
DOCTOR(  
  doctor_id PK,  
  doctor_name,  
  qualification,  
  specialization  
)
```

RELATIONSHIP DESCRIPTION

The Doctor entity is related to the Prescription entity.

- One Doctor can issue **many prescriptions (1:N)**
- Each Prescription is issued by exactly one Doctor

This relationship is implemented using the **doctor_id foreign key** in the Prescription table.

DETAILED LINE-BY-LINE EXPLANATION

Line 1: **CREATE TABLE IF NOT EXISTS doctor;**

This command creates the Doctor table if it does not already exist.

Line 2: **doctor_id INT NOT NULL;**

Defines doctor_id as an integer and ensures it cannot be empty.

Line 3: **doctor_name VARCHAR(50);**

Stores the doctor's name as a variable-length string.

Line 4: **qualification VARCHAR(260);**

Stores the doctor's qualifications. The larger size allows longer academic details.

Line 5: **specialization VARCHAR(50);**

Stores the doctor's medical specialization.

Line 6: **PRIMARY KEY (doctor_id);**

Sets doctor_id as the unique identifier for each doctor record.

Line 7: **INSERT IGNORE INTO doctor VALUES;**

Inserts multiple doctor records while ignoring duplicates.

Line 8: **(001, 'Mohamed Gamal', ...);**

Represents a sample record containing doctor ID, name, qualification, and specialization.

5.Prescription Entity Report

CONCEPTUAL DESIGN

Entity Name: Prescription

Description:

The Prescription entity stores the main information about a medical prescription issued by a doctor to a patient, including the patient name, issue date, and the doctor responsible for issuing the prescription. It represents the main record of a prescription before detailing its medicines.

ATTRIBUTES

- prescription_id
 - doctor_id
 - patient_name
 - issue_date
-

PRIMARY KEY

- prescription_id
-

CONSTRAINTS

- **PRIMARY KEY:** prescription_id (AUTO_INCREMENT)
- **NOT NULL:** patient_name, issue_date
- **FOREIGN KEY:** doctor_id REFERENCES Doctor(doctor_id)

RELATIONAL SCHEMA

```
PRESCRIPTION(  
  prescription_id PK,  
  doctor_id FK,  
  patient_name,  
  issue_date  
)
```

RELATIONSHIP DESCRIPTION

The Prescription entity is related to the Doctor entity.

- One Doctor can issue many Prescriptions (1:N)
- Each Prescription belongs to one Doctor

This relationship is implemented using the doctor_id foreign key in the Prescription table.

DETAILED LINE-BY-LINE EXPLANATION

Line 1: **CREATE TABLE IF NOT EXISTS Prescription;**
Creates the Prescription table if it does not already exist.

Line 2: **prescription_id INT PRIMARY KEY AUTO_INCREMENT;**
Defines a unique auto-generated ID for each prescription.

Line 3: **doctor_id INT;**
Stores the ID of the doctor issuing the prescription.

Line 4: **patient_name VARCHAR(100) NOT NULL;**
Stores the name of the patient.

Line 5: **issue_date DATE NOT NULL;**
Stores the date the prescription was issued.

Line 6: **FOREIGN KEY (doctor_id) REFERENCES Doctor(doctor_id);**
Links each prescription to a valid doctor.

6.Prescription_Details Entity Report

CONCEPTUAL DESIGN

Entity Name: Prescription_Details

Description:

The Prescription_Details entity stores the detailed line items of each prescription. It records which medicines are prescribed in a prescription and the quantity of each medicine. It acts as a linking entity between Prescription and Medicine.

ATTRIBUTES

- detail_id
 - prescription_id
 - medicine_id
 - quantity
-

PRIMARY KEY

- detail_id
-

CONSTRAINTS

- **PRIMARY KEY:** detail_id (AUTO_INCREMENT)
- **NOT NULL:** quantity
- **FOREIGN KEYS:**
 - prescription_id REFERENCES Prescription(prescription_id) **ON DELETE CASCADE**
 - medicine_id REFERENCES Medicine(medicine_id)

RELATIONAL SCHEMA

```
PRESCRIPTION_DETAILS(  
  detail_id PK,  
  prescription_id FK,  
  medicine_id FK,  
  quantity  
)
```

RELATIONSHIP DESCRIPTION

The Prescription_Details entity connects Prescription and Medicine.

- One Prescription can contain many medicines (1:N)
- One Medicine can appear in many prescriptions (1:N)
- This creates a **many-to-many (M:N)** relationship between Prescription and Medicine

The ON DELETE CASCADE constraint ensures that when a prescription is deleted, all related details are automatically removed.

DETAILED LINE-BY-LINE EXPLANATION

Line 1: **CREATE TABLE IF NOT EXISTS Prescription_Details;**
Creates the Prescription_Details table if it does not already exist.

Line 2: **detail_id INT PRIMARY KEY AUTO_INCREMENT;**
Defines a unique ID for each record.

Line 3: **prescription_id INT;**
Links the record to a prescription.

Line 4: **medicine_id INT;**
Links the record to a medicine.

Line 5: **quantity INT NOT NULL;**

Stores how many units of medicine are prescribed.

Line 6: **FOREIGN KEY (prescription_id) REFERENCES Prescription(prescription_id) ON DELETE CASCADE;**

Ensures relationship with Prescription and deletes details automatically if prescription is deleted.

Line 7: **FOREIGN KEY (medicine_id) REFERENCES Medicine(medicine_id);**

Ensures the medicine exists in the Medicine table.

7.Bill Entity Report

CONCEPTUAL DESIGN

Entity Name: Bill

Description:

The Bill entity stores financial information generated after a patient purchases prescribed medicines. It includes tax, discount, and total amount.

ATTRIBUTES

- bill_id
 - tax
 - discount
 - total
-

PRIMARY KEY

- bill_id
-

CONSTRAINTS

- **PRIMARY KEY:** bill_id
 - **AUTO_INCREMENT:** bill_id
 - **NOT NULL:** bill_id
-

RELATIONAL SCHEMA

```
BILL(  
  bill_id PK,  
  tax,  
  discount,  
  total  
)
```

RELATIONSHIP DESCRIPTION

The Bill entity is related to the Prescription entity.

- One Bill can be linked to **many prescriptions (M:1 relationship conceptually grouped)**
 - The billing process aggregates prescription-related costs into a single bill record
-

DETAILED LINE-BY-LINE EXPLANATION

Line 1: **CREATE TABLE IF NOT EXISTS Bill;**
Creates the Bill table if it does not already exist.

Line 2: **bill_id INT NOT NULL AUTO_INCREMENT;**
Defines bill_id as an integer and automatically generates a unique value for each record.

Line 3: **tax DECIMAL(5,2);**
Stores tax values with 2 decimal precision.

Line 4: **discount DECIMAL(5,2);**
Stores discount values with 2 decimal precision.

Line 5: **total DECIMAL(10,2);**
Stores the final bill amount with higher precision due to larger values.

Line 6: **PRIMARY KEY (bill_id);**
Ensures each bill has a unique identifier.

Line 7: **INSERT IGNORE INTO Bill VALUES;**
Inserts bill records while ignoring duplicate entries.

Line 8: **(1,2,0.5,0.15,727.75);**
Represents a sample bill record containing tax, discount, and final total.

TRIGGERS REPORT

CONCEPTUAL DESIGN

Entity Name: After_Prescription_Detail_Insert Trigger

Description:

This trigger automatically recalculates the total cost of a prescription whenever a new record is inserted into the Prescription_Details table. It ensures that the Bill table is always synchronized with the latest prescription items and their prices.

TRIGGER TYPE

- AFTER INSERT Trigger
 - Row-Level Trigger
-

TRIGGER LOGIC SUMMARY

- Trigger activates after inserting a record into Prescription_Details
 - It recalculates the total cost of the affected prescription
 - Calculation is based on: medicine price $\times$ quantity
 - It sums all items belonging to the same prescription_id
 - The Bill table is updated using the new total
-

DETAILED LINE-BY-LINE EXPLANATION

Line 1: **DROP TRIGGER IF EXISTS After_Prescription_Detail_Insert;**
Removes the trigger if it already exists to avoid duplication errors.

Line 2: **DELIMITER //**
Changes SQL delimiter to allow multi-line trigger definition.

Line 3: **CREATE TRIGGER After_Prescription_Detail_Insert**
Creates a trigger that executes automatically after insert operation.

Line 4: **AFTER INSERT ON PRESCRIPTION_DETAILS**

Specifies that the trigger runs after inserting a new prescription detail.

Line 5: **FOR EACH ROW**

Ensures the trigger runs for every inserted row individually.

Line 6: **DECLARE new_total DECIMAL(10,2);**

Declares a variable to store the calculated total value.

Line 7: **SELECT COALESCE(SUM(m.price * pd.quantity), 0) INTO new_total**

Calculates total cost by multiplying medicine price by quantity for all related prescription details. COALESCE ensures the result is 0 if no matching rows exist.

Line 8: **FROM PRESCRIPTION_DETAILS pd**

Uses Prescription_Details table as base.

Line 9: **JOIN MEDICINE m ON m.medicine_id = pd.medicine_id**

Joins Medicine table to retrieve price for each medicine.

Line 10: **WHERE pd.prescription_id = NEW.prescription_id**

Filters calculation to only the affected prescription using NEW.prescription_id.

Line 11: **UPDATE Bill**

Updates the Bill table with new calculated value.

Line 12: **SET total = new_total**

Assigns the computed total to the bill.

Line 13: **WHERE prescription_id = NEW.prescription_id;**

Ensures only the correct bill linked to that prescription is updated.

SUMMARY

The trigger ensures automatic recalculation of prescription totals whenever a new medicine is added, maintaining full consistency between prescription details and billing data.

TESTING QUERIES REPORT

CONCEPTUAL DESIGN

Entity Name: System Testing Queries

Description:

These queries are used to test the correctness of the database system, verify relationships between tables, and ensure that triggers and constraints are working properly.

TESTING PURPOSE

- Verify correct data insertion in all tables
 - Ensure relationships between tables work properly
 - Test trigger execution and automatic updates
 - Validate data consistency across the system
-

TESTING QUERIES SUMMARY

- SELECT queries for viewing stored data
 - INSERT queries to test trigger behavior
 - JOIN queries to verify relationships
 - Aggregation queries to validate calculations
-

DETAILED LINE-BY-LINE EXPLANATION

Line 1: **SELECT * FROM MEDICINE LIMIT 5;**

Displays a sample of medicine records to verify correct data insertion.

Line 2: **SELECT * FROM DISEASE LIMIT 5;**

Displays sample disease records from the system.

Line 3: **SELECT * FROM DOCTOR LIMIT 5;**

Shows sample doctor records to confirm correct insertion.

Line 4: **SELECT * FROM Prescription;**

Retrieves all prescription records stored in the system.

Line 5: **SELECT * FROM Prescription_Details;**

Displays all prescription line items (medicine + quantity).

Line 6: **SELECT * FROM Bill;**

Displays billing data before testing trigger effects.

Line 7: **INSERT INTO Prescription_Details (...)**

Adds a new prescription detail to test trigger activation.

Line 8: **SELECT * FROM Bill WHERE prescription_id = 1;**

Checks if the bill total has been updated after trigger execution.

Line 9: **SELECT SUM(m.price * pd.quantity) AS correct_total**

Manually calculates the correct total for comparison.

Line 10: **JOIN MEDICINE m ON pd.medicine_id = m.medicine_id**

Links medicine prices with prescription details for calculation.

Line 11: **WHERE pd.prescription_id = 1;**

Filters calculation for a specific prescription only.

Line 12: **Final comparison query**

Ensures that system-calculated total matches manual calculation.

SUMMARY

These queries validate system correctness, ensure trigger functionality, and confirm that all relationships and calculations in the database are working as expected.

REQUIRED QUERIES REPORT

CONCEPTUAL DESIGN

Entity Name: Database Query Requirements

Description:

These queries were specifically designed to meet instructor requirements, focusing on filtering, aggregation, and relationship validation across multiple entities.

QUERY PURPOSE

- Retrieve doctors based on specialization
 - Filter diseases by category
 - Analyze medicine usage statistics
 - Demonstrate relational database queries
-

DETAILED QUERIES EXPLANATION

Line 1: **SELECT** doctor_Name, qualification, specialization
FROM doctor
WHERE LOWER(specialization) LIKE '%cardio%';

Retrieves doctors whose specialization is related to cardiology using pattern matching.

Line 2: **SELECT** disease_id, disease_name
FROM DISEASE
WHERE disease_type = 'Deficiency Disease'
ORDER BY disease_name;

Displays all deficiency diseases sorted alphabetically.

```
Line 3: SELECT m.medicine_name, SUM(pd.quantity) AS total_quantity_sold
FROM PRESCRIPTION_DETAILS pd
JOIN MEDICINE m ON pd.medicine_id = m.medicine_id
JOIN Prescription p ON pd.prescription_id = p.prescription_id
WHERE p.issue_date BETWEEN '2023-01-01' AND '2023-12-31'
GROUP BY m.medicine_id, m.medicine_name
HAVING SUM(pd.quantity) = (
SELECT MAX(x.total_qty)
FROM (
SELECT SUM(pd2.quantity) AS total_qty
FROM PRESCRIPTION_DETAILS pd2
JOIN Prescription p2 ON pd2.prescription_id = p2.prescription_id
WHERE p2.issue_date BETWEEN '2023-01-01' AND '2023-12-31'
GROUP BY pd2.medicine_id
) x
);
```

Finds the most frequently sold medicine in 2023 by calculating total quantity and handling ties correctly.

SUMMARY

These queries demonstrate advanced SQL usage including filtering, joins, grouping, and subqueries to satisfy instructor requirements.

CONCLUSION

The Medicine Inventory Management System was successfully designed and implemented using a relational database structure. The project included the creation of entities, relationships, constraints, triggers, and SQL queries to ensure efficient data management and integrity. Testing queries were also performed to verify the correctness and functionality of the database components.